

day01- 文档加载（二）

一、系统架构

1.1 核心模块

索引：文档加载→文本分块→向量嵌入→向量存储。

检索：查询嵌入→相似度检索→提取上下文。

生成：提示工程→LLM 调度→输出后处理。

服务接口：REST API 端点→流式响应→监控指标。

1.2 核心框架

组件	技术选择	特点
嵌入模型	SentenceTransformers	句子级向量表示，可用于高效计算相似性。
向量数据库	Chroma	轻量级向量数据库，简单易用。支持持久化存储与元数据管理。
向量数据库	FAISS	高效的相似度搜索库，支持多种搜索算法。不具备持久化能力，需要手动持久化。
向量数据库	Milvus	支持大规模数据集的存储和检索，适合大规模生产级应用。提供完整的数据库功能，支持数据管理，持久化存储。
LLM运行时	vLLM	高性能推理，连续批处理。
服务框架	FastAPI	异步支持，自动生成 OpenAPI 文档。

1.3 环境准备

```
1 conda create -n rag python=3.12
2 conda activate rag
```

安装依赖：

```
1 pip install ipykernel
2 pip install langchain langchain-openai langchain_community unstructured markdown
  python-docx sseclient-py ragas langchain_huggingface sentence-transformers
3 pip install mineru[all] -i https://pypi.tuna.tsinghua.edu.cn/simple
4 pip install minio pymilvus pymilvus[model]
```

二、文档加载

2.1 普通文档加载

数据源可能来自多种格式的文件，如文本文档、Markdown，PDF等，首先需要对各种格式的文件进行处理，将其转化为可用的格式。

`langchain_community.document_loaders` 中提供了多种格式的文档加载器，包括：

- TextLoader（文本文档加载）
- UnstructuredMarkdownLoader（Markdown加载）
- PyPDFLoader（PDF加载）
- UnstructuredWordDocumentLoader（Word文档加载）
- WebBaseLoader（网站HTML加载）

等。

1) 加载文本文档

```
1 """
2 TextLoader会将一个文档文件加载为一个Document对象，该对象有两个属性：
3     metadata:      存储该文档的来源路径等元数据
4     page_content:   存储文档的内容
5 """
6
7 from langchain_community.document_loaders import TextLoader
8
9 text_documents = TextLoader("knowledge_base/sample.txt", encoding="utf-8").load()
10 print(text_documents)
```

2) 加载Markdown

```
1 """
2 UnstructuredMarkdownLoader可用于加载Markdown文件
3     mode: 加载模式
4         "single"    返回单个Document对象
5         "elements"  按标题等元素切分文档
6     strategy: 加载策略
7         "fast"      快速粗粒度加载
8         "hi_res"    细粒度加载，按标题层级、列表项、表格等结构细分
9 """
10
11 from langchain_community.document_loaders import UnstructuredMarkdownLoader
12
13 md_documents = UnstructuredMarkdownLoader(
14     "knowledge_base/sample.md",
15     mode="elements",
16     strategy="fast",
17 ).load()
18 print(md_documents)
```

3) 加载PDF

使用 PyPDFLoader 解析文档

```
1 """
2 PyPDFLoader
3     支持页级拆分，每一页作为一个Document返回
4     支持提取图像、提取布局
```

```

5         extraction_mode: 提取模式
6         "plain"         提取纯文本
7         "layout"        提取布局
8     """
9     from langchain_community.document_loaders import PyPDFLoader
10
11     pdf_documents = PyPDFLoader(
12         "knowledge_base/sample.pdf",
13         extraction_mode="layout",
14     ).load()
15     print(pdf_documents)

```

4) 加载Word文档

```

1     """
2     UnstructuredWordDocumentLoader
3         适用于 .docx 和 .doc 文件
4         mode: 加载模式
5             "single"    返回单个Document对象
6             "elements"  按标题等元素切分文档
7         strategy: 加载策略
8             "fast"      快速粗粒度加载
9             "hi_res"    细粒度加载, 按结构细分
10    """
11
12    from langchain_community.document_loaders import UnstructuredWordDocumentLoader
13
14    word_documents = UnstructuredWordDocumentLoader(
15        "knowledge_base/sample.docx",
16        mode="elements",
17        strategy="fast",
18    ).load()
19    print(word_documents)

```

5) 通过网址链接加载HTML

```

1     """
2     WebBaseLoader
3         适用于网页
4         web_paths: 网址序列
5         bs_kwargs: 传给 BeautifulSoup 的解析参数
6             parse_only 只提取指定标签的元素
7     """
8     import bs4
9     from langchain_community.document_loaders import WebBaseLoader
10
11    web_documents = WebBaseLoader(
12        web_paths=(
13            "https://news.sina.com.cn/c/x1/2025-09-07/doc-infprmw0510979.shtml",
14        ),
15        bs_kwargs={"parse_only": bs4.SoupStrainer(id="article")}, # 只提取正文区域
16    ).load()
17    print(web_documents)

```

6) 封装函数, 加载文件夹中多种文件类型

```

1     from langchain_community.document_loaders import (
2         TextLoader,

```

```

3     UnstructuredMarkdownLoader,
4     UnstructuredWordDocumentLoader
5 )
6
7 def load_documents():
8     """
9     加载多种类型的文档，包括text、markdown、PDF和Word文档
10
11     Returns:
12         list: 包含所有加载文档的列表
13     """
14     # 加载文本文件
15     text_documents = TextLoader(
16         "knowledge_base/sample.txt",
17         encoding="utf8"
18     ).load()
19
20     # 加载Markdown文件
21     md_documents = UnstructuredMarkdownLoader(
22         "knowledge_base/sample.md"
23     ).load()
24
25     # 加载Word文档
26     word_documents = UnstructuredWordDocumentLoader(
27         "knowledge_base/sample.docx"
28     ).load()
29
30     # 返回所有文档的列表
31     return text_documents + md_documents + word_documents
32
33 documents = load_documents()

```

7) 文本清洗

载文档后的清洗，是决定后续模型（如RAG应用）效果的关键一步，有句话叫“垃圾进，垃圾出”。

清洗工作通常分为两个层面：**文档级别**的清洗和**文本内容**的清洗。

第一层：文档级别清洗

在深入处理文本内容前，应先处理文档结构本身的问题。

1. **去除页眉、页脚和页码**：这些重复信息会增加噪声，应在处理早期移除。
2. **规范化处理表格**：表格包含结构化信息，简单的文本提取会丢失上下文。应将其作为独立对象处理，保留行列结构。
3. **语义化分块**：不建议机械地按固定字数切分文本。更优的做法是按**章节**、**标题**等语义边界进行分块，以保持内容的完整性。

第二层：文本内容清洗

这是清洗的核心，目标是去除噪声、统一格式。

- **去除噪声字符**：移除HTML标签、乱码、表情符号、特殊符号（如 ★）以及多余的空白字符。
- **文本标准化**：将文本统一转换为小写，并修正错误的Unicode字符。
- **文本规范化**：包括**词形还原**（如将"running"还原为"run"）和**拼写检查**。

第三层：高级文本清洗（可选）

根据任务需求，可进行更深入的处理。

- **分词 (Tokenization)**: 将文本切分为独立的单词或子词单元。
- **去除停用词 (Stopword Removal)**: 移除“的”、“了”、“the”、“is”等高频但无实际意义的词。
- **词干提取 (Stemming)**: 将单词简化为词干, 如将"running"变为"run"。

使用的 `unstructured` 库本身就提供了强大的清洗功能。可以直接利用其内置的清洁器 (cleaners) 。

例如, 在 `load_documents` 函数之后, 可以这样处理文本:

```
1  from unstructured.cleaners.core import (
2      clean,
3      clean_extra_whitespace,
4      replace_unicode_quotes,
5      clean_bullets,
6      clean_dashes,
7      clean_trailing_punctuation,
8  )
9
10 def clean_text(text: str) -> str:
11     # 1. 修复 Unicode 引号
12     text = replace_unicode_quotes(text)
13     # 2. 清理项目符号 (如 •、- 等)
14     text = clean_bullets(text)
15     # 3. 清理破折号 (统一为普通连字符)
16     text = clean_dashes(text)
17     # 4. 移除末尾标点 (如句号、问号等)
18     text = clean_trailing_punctuation(text)
19     # 5. 压缩多余空白字符 (包括空格、换行、制表符)
20     text = clean_extra_whitespace(text)
21     # 6. 转为小写 (可选)
22     text = text.lower()
23     return text
24
25 # 应用到所有文档
26 for doc in documents:
27     doc.page_content = clean_text(doc.page_content)
```

也可以编写自定义清洗函数, 并应用到每个文档元素上

2.2 MinerU 加载 PDF 文档

MinerU 是一个开源的 PDF 解析工具, 能够将 PDF 文档转换为 Markdown 格式, 保留文档结构和图片。

在线体验: <https://mineru.net/OpenSourceTools/Extractor>

官网地址: <https://opendatalab.github.io/MinerU/zh/>

2.2.1 模型部署

解析后端	pipeline	*-engine		*-http-client	
		hybrid	vlm	hybrid	vlm
后端特性	兼容性好	硬件配置要求较高		适用于OpenAI兼容服务器 ²	
精度指标 ¹	86.47	95.39 (high) 95.26 (medium)	95.30	95.39 (high) 95.26 (medium)	95.30
操作系统	Linux ³ / Windows ⁴ / macOS ⁵				
纯CPU平台支持	✓	✗		✓	
GPU加速支持	Volta及以后架构GPU或Apple Silicon				不需要
显存最低要求	4GB	8GB		2GB	
内存要求	最低16GB以上,推荐32GB以上			最低16GB	
磁盘空间要求	20GB以上,推荐使用SSD			至少2GB	
python版本	3.10-3.13				

安装：

```
1 pip install mineru[all] -i https://pypi.tuna.tsinghua.edu.cn/simple
```

命令行用法：

如果您的设备满足上表中GPU加速的条件，可以使用简单的命令行进行文档解析：

```
1 mineru -p <input_path> -o <output_path>
```

如果您的设备不满足GPU加速条件，可以指定后端为 `pipeline`，以在纯CPU环境下运行：

```
1 #mineru -p <input_path> -o <output_path> -b pipeline
2
3 mineru -p D:\\sample.pdf -o D:\\result -b pipeline
```

当前 `mineru` 支持本地 `PDF / 图片 / DOCX / PPTX / XLSX` 文件或目录输入，并可通过命令行、API、WebUI、`mineru-router` 等多种方式进行文档解析。

```
(rag) D:\>mineru -p D:\\sample.pdf -o D:\\result -b pipeline
2026-07-27 22:00:58.023 | INFO | mineru.cli.client:run_orchestrated_cli:953 - Started local mineru-api at http
://127.0.0.1:59177
2026-07-27 22:01:09.149 | INFO | __main__:create_app:236 - Request concurrency limited to 3
Start MinerU FastAPI Service: http://127.0.0.1:59177
API documentation: http://127.0.0.1:59177/docs
INFO: Started server process [15156]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: Uvicorn running on http://127.0.0.1:59177 (Press CTRL+C to quit)
2026-07-27 22:01:10.159 | INFO | mineru.cli.client:run_planned_task:832 - Submitting batch 1/1 | 1 document, 3
6 pages in this batch | 36 pages total | task#1 [sample]
2026-07-27 22:01:24.153 | INFO | mineru.backend.pipeline.pipeline_analyze:doc_analyze_streaming:209 - Pipeline
processing-window multi-file run. doc_count=1, total_pages=36, window_size=64, total_batches=1
2026-07-27 22:01:41.235 | INFO | mineru.backend.pipeline.pipeline_analyze:doc_analyze_streaming:264 - Pipeline
processing window batch 1/1: 36/36 pages, batch_pages=36, doc_slices=doc0:1-36
2026-07-27 22:01:41.240 | INFO | mineru.backend.pipeline.pipeline_analyze:batch_image_analyze:364 - GPU Memory
: 1 GB, Batch Ratio: 1.
2026-07-27 22:01:41.241 | INFO | mineru.backend.pipeline.model_init:__init__:239 - DocAnalysis init, this may
take some times.....
2026-07-27 22:01:47.986 | INFO | mineru.backend.pipeline.model_init:__init__:291 - DocAnalysis init done!
2026-07-27 22:01:47.991 | INFO | mineru.backend.pipeline.pipeline_analyze:custom_model_init:83 - model init co
st: 6.750191926956177
Layout Predict: 100%| 36/36 [02:23<00:00, 3.99s/it]
Table orientation: 100%| 49/49 [01:14<00:00, 1.53s/it]
Table-ocr det: 100%| 19/19 [00:52<00:00, 2.75s/it]
Table-ocr rec ch: 100%| 1110/1110 [00:51<00:00, 21.51it/s]
Table-wireless Predict: 100%| 19/19 [00:04<00:00, 4.08it/s]
Table-wired Predict: 100%| 19/19 [00:32<00:00, 1.72s/it]
OCR-det ch: 100%| 147/147 [01:45<00:00, 1.40it/s]
Seal Predict: 0it [00:00, ?it/s]
OCR-rec Predict: 100%| 2/2 [00:00<00:00, 4.72it/s]
Processing pages: 100%| 36/36 [00:03<00:00, 9.06it/s]
2026-07-27 22:09:41.715 | INFO | mineru.cli.client:run_planned_task:883 - Completed batch 1/1 | Processed 36/3
6 pages | 1 of 1 batch finished | task#1 [sample]
```

- 1 #如果本地有GPU,这是默认使用的是混合模式(hybrid:pipeline+vlm)
- 2 mineru -p input.pdf -o output_dir/ --source local

参数说明:

参数	说明
-p	输入 PDF 文件路径
-o	输出目录
--source local	使用本地已下载模型（避免每次下载）

输出目录结构:

```
1 output_dir/
2   └── 文件名/
3       └── hybrid_auto/
4           ├── 文件名.md      # 转换后的 Markdown
5           └── images/        # 提取的图片
6               ├── image_0.png
7               └── image_1.png
```

详细解释参考: https://opendatalab.github.io/MinerU/zh/reference/output_files/

2.2.2 PDF转Markdown

1 pathlib.Path

Python 3.4+ 引入的面向对象路径处理库, 比 os.path 更优雅:

```
1 from pathlib import Path
2
3 # 创建 Path 对象
4 path = Path("D:/documents/万用表的使用.pdf")
5
6 # 常用属性
```

```

7  path.name      # "万用表的使用.pdf" - 完整文件名
8  path.stem     # "万用表的使用" - 不含扩展名的文件名
9  path.suffix   # ".pdf" - 扩展名
10 path.parent   # Path("D:/documents") - 父目录
11
12 # 常用方法
13 path.exists()  # True/False - 文件是否存在
14 path.is_file() # True/False - 是否为文件
15 path.is_dir()  # True/False - 是否为目录
16
17 # 路径拼接（使用 / 运算符）
18 output_path = path.parent / "output" / "result.md"

```

2 subprocess 子进程调用

subprocess 模块用于调用外部命令行工具：

```

1  import subprocess
2
3  # 基础用法: run() - 同步执行，等待完成
4  result = subprocess.run(
5      ["mineru", "-p", "input.pdf", "-o", "output/"],
6      capture_output=True, # 捕获输出
7      text=True,           # 文本模式
8      check=True           # 非零返回码抛出异常
9  )
10 print(result.stdout)
11
12 # 高级用法: Popen() - 实时获取输出
13 proc = subprocess.Popen(
14     ["mineru", "-p", "input.pdf"],
15     stdout=subprocess.PIPE, # 捕获标准输出
16     stderr=subprocess.STDOUT, # 合并错误到标准输出
17     text=True,
18     bufsize=1 # 行缓冲，实时输出
19 )
20
21 # 逐行读取输出
22 for line in proc.stdout:
23     print(line.rstrip())
24
25 # 等待完成
26 return_code = proc.wait()

```

关键参数说明：

参数	说明
<code>stdout=subprocess.PIPE</code>	捕获标准输出
<code>stderr=subprocess.STDOUT</code>	将错误流合并到标准输出
<code>text=True</code>	文本模式（自动解码为字符串）
<code>encoding="utf-8"</code>	指定编码
<code>errors="replace"</code>	遇到无法解码的字符时替换（避免崩溃）
<code>env=os.environ.copy()</code>	传递当前环境变量
<code>bufsize=1</code>	行缓冲，每行立即输出

3 实现过程

```
1 import subprocess
2 import logging
3 import time
4 from pathlib import Path
5
6
7 logging.basicConfig(
8     level=logging.DEBUG,
9     format='%(asctime)s - %(name)s - %(levelname)s - %(message)s'
10 )
11 logger = logging.getLogger("mineru")
12
13 def execute_mineru(pdf_path_obj: Path, output_dir_obj: Path) -> int:
14     """
15     执行 MinerU 命令
16
17     Args:
18         pdf_path_obj: PDF 文件 Path 对象
19         output_dir_obj: 输出目录 Path 对象
20
21     Returns:
22         命令返回码（0 表示成功）
23     """
24     if not Path(pdf_path_obj).exists():
25         print(f"警告：测试文件不存在：{pdf_path_obj}")
26         print("请修改 pdf_path_obj 为有效的 PDF 文件路径")
27     else:
28         pdf_path_obj = Path(pdf_path_obj)
29         if not pdf_path_obj.exists():
30             print(f"PDF 文件不存在：{pdf_path_obj}")
31
32     # 获取输出目录
33
34     output_dir_obj = Path(output_dir_obj)
35     cmd = [
36         "mineru",
37         "-p", str(pdf_path_obj),
38         "-o", str(output_dir_obj),
39         "-b", "pipeline" # 使用本地模型
40     ]
```

```

41
42     start_ts = time.time()
43
44     # 2. 调用命令行工具
45     proc = subprocess.Popen(
46         cmd,
47         stdout=subprocess.PIPE,      # 捕获标准输出
48         stderr=subprocess.STDOUT,    # 合并错误到 stdout
49         text=True,                   # 文本模式
50         encoding="utf-8",
51         errors="replace",             # 遇到乱码时替换
52         bufsize=1,                   # 行缓冲（实时输出）
53     )
54
55     # 3. 实时输出日志
56     for line in proc.stdout:
57         logger.debug(f"[mineru] {line.rstrip()}")
58
59     # 4. 等待命令完成
60     return_code = proc.wait()
61
62     elapsed = time.time() - start_ts
63     if return_code == 0:
64         logger.info(f"转换完成, 耗时: {elapsed:.2f} 秒, return_code:{return_code}")
65     else:
66         logger.error("转换失败")
67     return return_code
68
69 def _get_output_path(pdf_path_obj: Path, output_dir_obj: Path) -> str:
70     """
71     获取转换结果路径
72
73     MinerU 输出目录结构:
74     output_dir/
75         └── 文件名/
76             ├── auto/
77                 ├── 文件名.md
78                 └── images/
79
80     Args:
81         pdf_path_obj: PDF 文件 Path 对象
82         output_dir_obj: 输出目录 Path 对象
83
84     Returns:
85         Markdown 文件路径字符串
86     """
87     file_stem = pdf_path_obj.stem
88     md_path = output_dir_obj / file_stem / "auto" / f"{file_stem}.md"
89     return str(md_path)
90
91 if __name__ == "__main__":
92     test_pdf_path = Path(r"D:\project\knowledge_base\sample.pdf")
93     test_output_dir = Path(r"D:\project\knowledge_base\result")
94     return_code = execute_mineru(test_pdf_path, test_output_dir)
95     if return_code == 0:
96         md_path = _get_output_path(test_pdf_path, test_output_dir)
97         print(f"转换成功, 结果保存在: {md_path}")

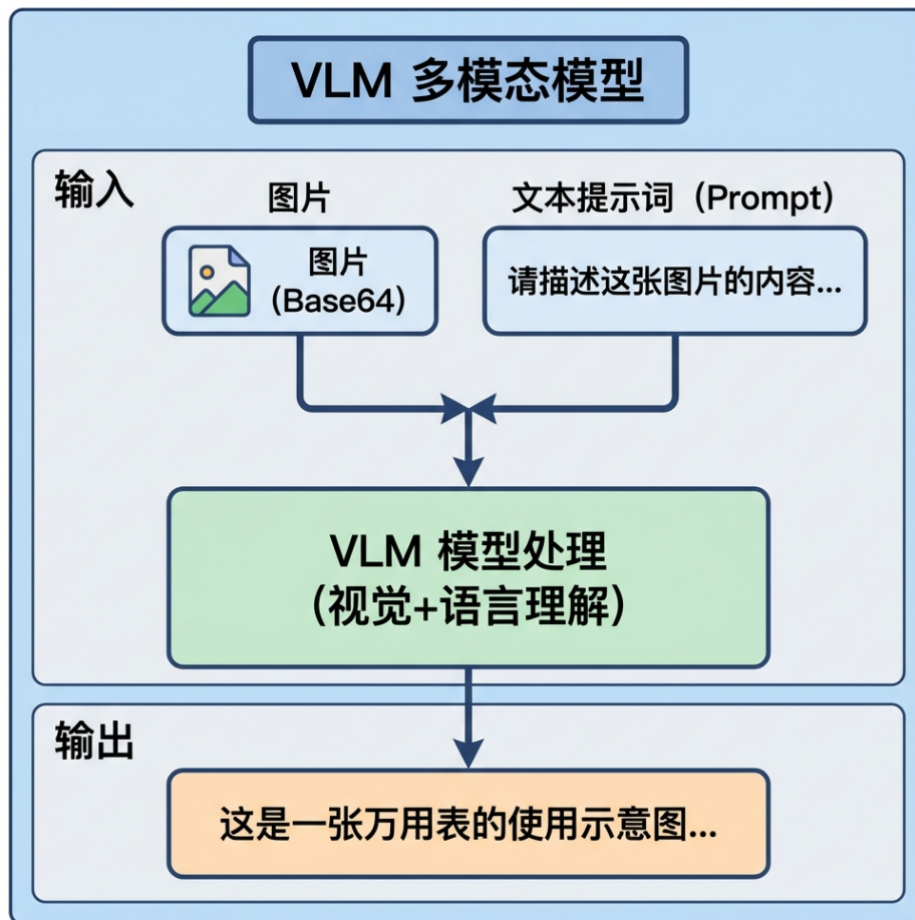
```

2.2.3 图片处理

1 Vision-Language Model (VLM)

VLM（视觉-语言模型） 是能够同时处理图像和文本的多模态大模型。

工作原理：



API 调用示例（OpenAI 兼容格式）：

```
1  from openai import OpenAI
2  import base64
3
4  # 初始化客户端
5  client = OpenAI(
6      api_key="your-api-key",
7      base_url="https://api.example.com/v1"
8  )
9
10 # 图片转 Base64
11 with open("image.jpg", "rb") as f:
12     base64_image = base64.b64encode(f.read()).decode("utf-8")
13 doc_title = "markdown标题"
14 section_heading = "当前段落标题"
15 pre_text = "图片之前段落内容"
16 post_text = "图片之后段落内容"
17
18 # 构造 Prompt
19 context_parts = []
20 if section_heading:
21     context_parts.append(f"所属章节标题: {section_heading}")
22 if pre_text:
```

```

23     context_parts.append(f"图片上文: {pre_text}")
24     if post_text:
25         context_parts.append(f"图片下文: {post_text}")
26
27     context_info = "\n".join(context_parts) if context_parts else "无可用上下文"
28
29     # 调用 VLM
30     response = client.chat.completions.create(
31         model="qwen-vl-plus", # 视觉模型
32         messages=[
33             {
34                 "role": "user",
35                 "content": [{
36                     "type": "text",
37                     "text": f""任务: 为Markdown文档中的图片生成一个简短的中文标题。
38 背景信息:
39 1. 所属文档标题: "{doc_title}"
40 2. 图片上下文:
41     {context_info}
42 请结合图片视觉内容和上述上下文信息, 用中文简要总结这张图片的内容,
43 生成一个精准的中文标题(不要包含"图片"二字)。""",
44                 },
45                 {
46                     "type": "image_url",
47                     "image_url": {
48                         "url": f"data:image/jpeg;base64,{base64_image}"
49                     }
50                 }
51             ]
52         },
53         max_tokens=100
54     )
55
56     summary = response.choices[0].message.content

```

2 Markdown 图片语法

标准图片语法:

```

1     ![alt文本](图片路径)
2
3     # 示例
4     ![万用表正面图](images/image_0.png)
5     

```

正则表达式匹配:

```

1  import re
2
3  md_content = "这是一段文字 ![图片描述](images/photo.jpg) 后面的文字"
4
5  # 匹配所有图片
6  pattern = r"!\[.*?\]\(.*?\)"
7  matches = re.findall(pattern, md_content)
8  # [('图片描述', 'images/photo.jpg')]
9
10 # 匹配特定图片文件名
11 image_filename = "photo.jpg"
12 specific_pattern = r"!\[.*?\]\(.*?" + re.escape(image_filename) + r".*?\)"
13 # 注意：使用 re.escape() 转义文件名中的特殊字符

```

3 Base64 编码

概念： 将二进制数据编码为 ASCII 字符串，常用于在文本协议中传输二进制数据。

```

1  import base64
2
3  # 编码
4  with open("image.png", "rb") as f:
5      binary_data = f.read()
6      base64_string = base64.b64encode(binary_data).decode("utf-8")
7
8  # 解码
9  binary_data = base64.b64decode(base64_string)

```

在 VLM 中的应用：

```

1  原始图片（二进制）
2      |
3      ▼ base64.b64encode()
4  Base64 字符串: "iVBORw0KGgoAAAANSUhEUgAA..."
5      |
6      ▼ 拼接为 Data URL
7  "data:image/jpeg;base64,iVBORw0KGgoAAAANSUhEUgAA..."
8      |
9      ▼ 作为 API 请求的一部分发送
10  VLM 模型处理

```

4 图片处理业务流程

- 读取并解析 Markdown 文档中的图片引用
- 利用 VLM 为每张图片生成语义化的中文摘要
- 将本地图片上传至 MinIO 对象存储
- 替换 Markdown 中的本地路径为远程 URL
- 生成处理后的新 Markdown 文件

处理前后对比

处理前 (原始 Markdown)：

```
1 # 万用表概述
2
3 万用表是一种多功能测量仪器。
4
5 
6
7 上图展示了万用表的正面外观。
```

处理后 (新 Markdown):

```
1 # 万用表概述
2
3 万用表是一种多功能测量仪器。
4
5 ![万用表数字显示屏及旋钮面板](http://minio:9000/knowledge-base/万用表的使用/image_0.png)
6
7 上图展示了万用表的正面外观。
```

2.2.4 minio使用

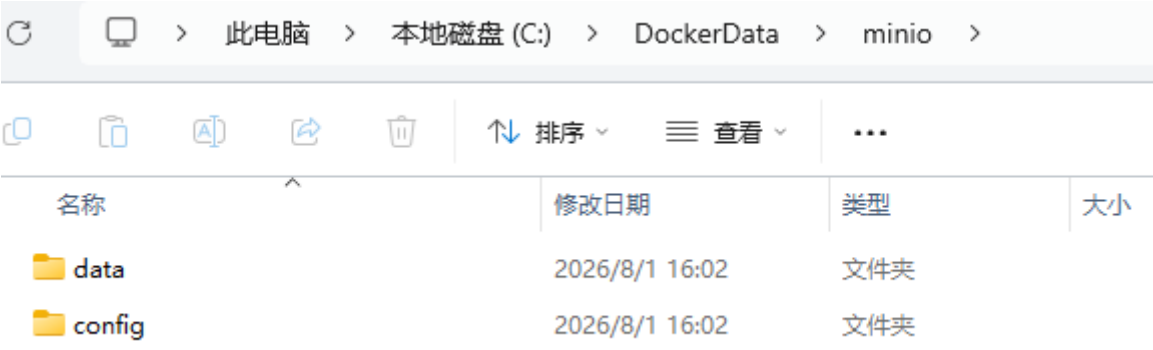
Docker Desktop部署minio

1、拉取镜像

```
docker pull minio/minio:RELEASE.2024-12-18T13-15-44Z
```

2、创建容器映射文件夹

```
创建文件夹 C:\DockerData\minio\data（数据）和 C:\DockerData\minio\config（配置）
```



3、创建容器

```
1 docker run -d ^
2   -p 9000:9000 ^
3   -p 9090:9090 ^
4   --name minio ^
5   --restart=always ^
6   -e "MINIO_ROOT_USER=minioadmin" ^
7   -e "MINIO_ROOT_PASSWORD=minioadmin" ^
8   -e "MINIO_SERVER_URL=http://localhost:9000" ^
9   -e "MINIO_BROWSER_REDIRECT_URL=http://localhost:9090" ^
10  -v D:\DockerData\minio\data:/data ^
11  minio/minio:RELEASE.2024-12-18T13-15-44Z server /data ^
12  --address ":9000" ^
13  --console-address ":9090"
```

```

14
15     # 稍微说一下
16     # -d 后台运行
17     # -p 9000:9000 前面的9000是打开本机的9000端口映射docker容器中MinIO的9000端口
18     # -p 9090:9090
19     # -e "MINIO_ROOT_USER=minioadmin" \ 改用户名
20     # -e "MINIO_ROOT_PASSWORD=minioadmin" \ 改密码
21     # D:\DockerData\minio\data:/data \ 挂载卷
22     # D:\DockerData\minio/config:/root/.minio \ 挂载卷
23     # --console-address ":9000" 指定控制台端口为静态端口
24     # -address ":9090" 指定API端口为静态端口

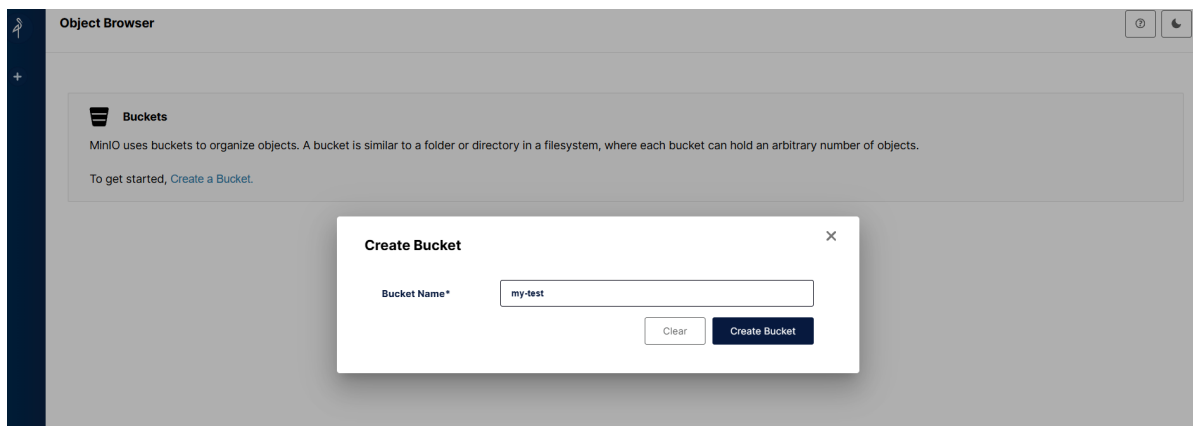
```

可以通过 <http://127.0.0.1:9090/login> 访问端口

账号密码：minioadmin/minioadmin

4、基本使用

创建好桶之后就可以直接上传图片了



代码实现图片上传：<https://docs.min.io/aistor/developers/sdk/python/>

```

1  # file_uploader.py MinIO Python SDK example
2  from minio import Minio
3  from minio.error import S3Error
4
5  def main():
6      # Create a client with the MinIO server playground, its access key
7      # and secret key.
8      client = Minio("127.0.0.1:9000",
9                    access_key="minioadmin",
10                   secret_key="minioadmin",
11                   secure=False
12                )
13
14     # The file to upload, change this path if needed
15     source_file = r"..\images\RAG.png"
16
17     # The destination bucket and filename on the MinIO server
18     bucket_name = "my-test"
19     destination_file = "rag-1.png"
20
21     # Make the bucket if it doesn't exist.
22     found = client.bucket_exists(bucket_name)
23     if not found:
24         client.make_bucket(bucket_name)
25         print("Created bucket", bucket_name)

```

```

26         else:
27             print("Bucket", bucket_name, "already exists")
28
29         # Upload the file, renaming it in the process
30         client.fput_object(
31             bucket_name, destination_file, source_file,
32         )
33         print(
34             source_file, "successfully uploaded as object",
35             destination_file, "to bucket", bucket_name,
36         )
37
38     if __name__ == "__main__":
39         try:
40             main()
41         except S3Error as exc:
42             print("error occurred.", exc)

```

5、MinIO 工具类

```

1     # minio_utils.py
2
3     import os
4     from minio import Minio
5
6     # MinIO 配置（支持环境变量覆盖）
7     MINIO_ENDPOINT = os.getenv("MINIO_ENDPOINT", "127.0.0.1:9000")
8     MINIO_ACCESS_KEY = os.getenv("MINIO_ACCESS_KEY", "minioadmin")
9     MINIO_SECRET_KEY = os.getenv("MINIO_SECRET_KEY", "minioadmin")
10    MINIO_BUCKET_NAME = os.getenv("MINIO_BUCKET_NAME", "knowledge-base")
11
12
13    # 初始化 MinIO 客户端
14    try:
15        minio_client = Minio(
16            MINIO_ENDPOINT,
17            access_key=MINIO_ACCESS_KEY,
18            secret_key=MINIO_SECRET_KEY,
19            secure=False # 本地开发不使用 HTTPS
20        )
21        # 确保 Bucket 存在
22        if not minio_client.bucket_exists(MINIO_BUCKET_NAME):
23            minio_client.make_bucket(MINIO_BUCKET_NAME)
24    except Exception as e:
25        print(f"MinIO initialization failed: {e}")
26        minio_client = None
27
28
29    def get_minio_client():
30        """获取 MinIO 客户端单例"""
31        return minio_client

```

关键设计点:

1. 环境变量支持

- 使用 `os.getenv()` 支持通过环境变量覆盖默认配置
- 便于在不同环境（开发/测试/生产）中切换

2. 单例模式

- 模块级别初始化，全局共享一个客户端实例
- `get_minio_client()` 返回已初始化的客户端

3. 自动创建 Bucket

- 初始化时检查 Bucket 是否存在
- 不存在则自动创建

2.2.5 完整过程

```
1  # md_img.py
2  import subprocess
3  import logging
4  import time
5  from pathlib import Path
6  from typing import Dict, List, Tuple, Set
7  import os
8  import re
9
10 import base64
11 from minio_utils import get_minio_client, MINIO_BUCKET_NAME
12
13 logging.basicConfig(
14     level=logging.DEBUG,
15     format='%(asctime)s - %(name)s - %(levelname)s - %(message)s'
16 )
17 logger = logging.getLogger("mineru")
18
19 image_extensions: Set[str] = {".jpg", ".jpeg", ".png", ".gif", ".bmp", ".webp"}
20 vl_model = "qwen3-vl-plus"
21 # =====
22 # Object Storage (MinIO)
23 # =====
24 # MinIO 服务端点
25 MINIO_ENDPOINT="127.0.0.1:9000"
26 # 访问密钥
27 MINIO_ACCESS_KEY="minioadmin"
28 # 私有密钥
29 MINIO_SECRET_KEY="minioadmin"
30 MINIO_SECURE: bool = False
31 # 存储桶名称
32 # MINIO_BUCKET_NAME=knowledge-base
33 # MINIO_BUCKET_NAME=knowledge-base-v1
34 #MINIO_BUCKET_NAME="knowledge-base"
35 def execute_mineru(pdf_path_obj: Path, output_dir_obj: Path) -> int:
36     """
37     执行 MinerU 命令
38
39     Args:
40         pdf_path_obj: PDF 文件 Path 对象
41         output_dir_obj: 输出目录 Path 对象
42
43     Returns:
44         命令返回码 (0 表示成功)
45     """
46     if not Path(pdf_path_obj).exists():
47         print(f"警告：测试文件不存在：{pdf_path_obj}")
```

```

48         print("请修改 pdf_path_obj 为有效的 PDF 文件路径")
49     else:
50         pdf_path_obj = Path(pdf_path_obj)
51         if not pdf_path_obj.exists():
52             print(f"PDF 文件不存在: {pdf_path_obj}")
53
54     # 获取输出目录
55
56     output_dir_obj = Path(output_dir_obj)
57     cmd = [
58         "mineru",
59         "-p", str(pdf_path_obj),
60         "-o", str(output_dir_obj),
61         "-b", "pipeline" # 使用本地模型
62     ]
63
64     start_ts = time.time()
65
66     # 2. 调用命令行工具
67     proc = subprocess.Popen(
68         cmd,
69         stdout=subprocess.PIPE, # 捕获标准输出
70         stderr=subprocess.STDOUT, # 合并错误到 stdout
71         text=True, # 文本模式
72         encoding="utf-8",
73         errors="replace", # 遇到乱码时替换
74         bufsize=1, # 行缓冲（实时输出）
75     )
76
77     # 3. 实时输出日志
78     for line in proc.stdout:
79         logger.debug(f"[mineru] {line.rstrip()}")
80
81     # 4. 等待命令完成
82     return_code = proc.wait()
83
84     elapsed = time.time() - start_ts
85     if return_code == 0:
86         logger.info(f"转换完成, 耗时: {elapsed:.2f} 秒, return_code:{return_code}")
87     else:
88         logger.error("转换失败")
89     return return_code
90
91 def _get_md_content_and_path(md_file_path_str:str) -> Tuple[str, Path, Path]:
92     """
93     读取 Markdown 文件内容并获取相关路径对象。
94
95     Args:
96         md_file_path_str: markdown 文件路径字符串。
97
98     Returns:
99         Tuple[str, Path, Path]:
100             - md_content: Markdown 文件的文本内容。
101             - md_path_obj: Markdown 文件的 Path 对象。
102             - images_dir_obj: 关联的 images 目录 Path 对象。
103
104     Raises:
105         ImageProcessingError: 当 'md_path' 为空时抛出。

```

```

106     """
107     logger.info("step_1: 读取 MD 内容")
108
109     md_path_obj = Path(md_file_path_str)
110     try:
111         with open(md_path_obj, "r", encoding="utf-8") as f:
112             md_content = f.read()
113     except IOError as e:
114         raise Exception(f"无法读取文件 {md_path_obj}: {e}")
115
116     # images 目录位于 md 文件同级目录下
117     images_dir_obj = md_path_obj.parent / "images"
118     return md_content, md_path_obj, images_dir_obj
119
120 def _scan_and_filter_images(
121     md_content: str,
122     images_dir_obj: Path,
123     allowed_extensions: set
124 ) -> List[Tuple[str, str, Tuple[str, str, str]]]:
125     """
126     扫描 images 目录，筛选出在 Markdown 内容中被引用的有效图片。
127
128     Args:
129         md_content (str): Markdown 文本内容。
130         images_dir_obj (Path): 图片目录路径对象。
131         allowed_extensions (set): 允许处理的图片扩展名集合。
132
133     Returns:
134         List[Tuple[str, str, Tuple[str, str, str]]]: 图片信息列表。
135     """
136     logger.info(f"step_2: 扫描图片目录: {images_dir_obj}")
137
138     target_images = []
139
140     # 遍历 images 目录下的所有文件
141     for image_filename in os.listdir(images_dir_obj):
142         # 检查扩展名是否在允许列表中
143         file_ext = os.path.splitext(image_filename)[1].lower()
144         if file_ext not in allowed_extensions:
145             continue
146
147         image_full_path = str(images_dir_obj / image_filename)
148
149         # 在 Markdown 内容中查找该图片的引用上下文
150         contexts_list = _find_image_contexts_in_md(md_content,
151 image_filename)
152
153         if not contexts_list:
154             logger.debug(f"图片 {image_filename} 未在文档中被引用，跳过。")
155             continue
156
157         # 取第一个引用处的上下文用于生成摘要
158         primary_context = contexts_list[0]
159         target_images.append((image_filename, image_full_path,
160 primary_context))
161
162     logger.info(f"找到 {len(target_images)} 张需要处理的有效图片。")
163     return target_images

```

```

162
163 def _find_image_contexts_in_md(
164     md_content: str,
165     image_filename: str,
166     max_chars: int = 100
167 ) -> List[Tuple[str, str, str]]:
168     """
169     基于 Markdown 语义结构查找图片的上下文。
170
171     策略:
172     1. 向上查找最近的标题行（# 开头的行）作为 section 标题。
173     2. 取标题到图片之间的完整段落作为上文。
174     3. 向下取图片后的 1-2 个完整段落作为下文。
175     4. 上文和下文分别不超过 max_chars 字符。
176
177     Args:
178         md_content (str): Markdown 文本内容。
179         image_filename (str): 要查找的图片文件名。
180         max_chars (int, optional): 上下文最大字符数。默认为 100。
181
182     Returns:
183         List[Tuple[str, str, str]]: 上下文列表。
184     """
185     lines = md_content.split("\n")
186
187     # 构建正则匹配图片引用行
188     image_pattern = re.compile(
189         r"!\[.*?\]\(.*?" + re.escape(image_filename) + r".*?\)"
190     )
191
192     contexts_list = []
193
194     for line_idx, line in enumerate(lines):
195         if not image_pattern.search(line):
196             continue
197
198         # 向上查找最近的标题
199         section_heading = ""
200         heading_line_idx = -1
201
202         for i in range(line_idx - 1, -1, -1):
203             if re.match(r"^#{1,6}\s+", lines[i]):
204                 section_heading = lines[i].strip()
205                 heading_line_idx = i
206             break
207
208         # 提取上文
209         pre_start = heading_line_idx + 1 if heading_line_idx >= 0 else 0
210         pre_lines = lines[pre_start:line_idx]
211         pre_paragraphs = _extract_paragraphs_with_limit(
212             pre_lines, max_chars, direction="backward"
213         )
214
215         # 向下查找下文边界
216         next_heading_idx = len(lines)
217         for i in range(line_idx + 1, len(lines)):
218             if re.match(r"^#{1,6}\s+", lines[i]):
219                 next_heading_idx = i

```

```

220             break
221
222         post_lines = lines[line_idx + 1:next_heading_idx]
223         post_paragraphs = _extract_paragraphs_with_limit(
224             post_lines, max_chars, direction="forward"
225         )
226
227         contexts_list.append((section_heading, pre_paragraphs,
228                                post_paragraphs))
229
230     return contexts_list
231
232 def _extract_paragraphs_with_limit(
233     lines: List[str],
234     max_chars: int,
235     direction: str = "forward"
236 ) -> str:
237     """
238     从给定的行列表中提取完整段落，总字符数不超过 max_chars。
239
240     Args:
241         lines (List[str]): 待提取的行列表。
242         max_chars (int): 最大字符数限制。
243         direction (str): "forward" 从前往后, "backward" 从后往前。
244
245     Returns:
246         str: 拼接后的段落文本。
247     """
248     # 将连续的非空行合并为一个段落
249     paragraphs = []
250     current_para = []
251
252     for line in lines:
253         stripped = line.strip()
254         if stripped == "":
255             if current_para:
256                 paragraphs.append("\n".join(current_para))
257                 current_para = []
258             else:
259                 # 跳过图片行
260                 if re.match(r"^!\\[.*?\\](.*?\\)$", stripped):
261                     if current_para:
262                         paragraphs.append("\n".join(current_para))
263                         current_para = []
264                     continue
265                 current_para.append(stripped)
266
267     if current_para:
268         paragraphs.append("\n".join(current_para))
269
270     paragraphs = [p for p in paragraphs if p.strip()]
271
272     if not paragraphs:
273         return ""
274
275     # backward 优先取靠近图片的段落
276     if direction == "backward":
277         paragraphs = list(reversed(paragraphs))

```

```

277
278     # 在字符数限制内尽量多取完整段落
279     selected = []
280     total_chars = 0
281
282     for para in paragraphs:
283         para_len = len(para)
284         if total_chars + para_len > max_chars and selected:
285             break
286         selected.append(para)
287         total_chars += para_len
288
289     if direction == "backward":
290         selected = list(reversed(selected))
291
292     return "\n\n".join(selected)
293
294 def _call_vlm_for_summary(
295     client,
296     model_name: str,
297     image_path: str,
298     doc_title: str,
299     context_tuple: Tuple[str, str, str]
300 ) -> str:
301     """
302     调用 VLM 接口生成图片描述。
303
304     Args:
305         client: OpenAI 客户端实例。
306         model_name (str): 模型名称。
307         image_path (str): 图片文件路径。
308         doc_title (str): 文档标题。
309         context_tuple (Tuple[str, str, str]): 上下文三元组。
310
311     Returns:
312         str: 生成的图片摘要。
313     """
314     # 读取并 Base64 编码
315     try:
316         with open(image_path, "rb") as image_file:
317             base64_image = base64.b64encode(image_file.read()).decode("utf-
318 8")
319
320     except IOError as e:
321         logger.error(f"无法读取图片文件 {image_path}: {e}")
322         return "图片读取失败"
323
324     section_heading, pre_text, post_text = context_tuple
325
326     # 构造 Prompt
327     context_parts = []
328     if section_heading:
329         context_parts.append(f"所属章节标题: {section_heading}")
330     if pre_text:
331         context_parts.append(f"图片上文: {pre_text}")
332     if post_text:
333         context_parts.append(f"图片下文: {post_text}")

```

```

333     context_info = "\n".join(context_parts) if context_parts else "无可用上下
文"
334
335     try:
336         response = client.chat.completions.create(
337             model=model_name,
338             messages=[
339                 {
340                     "role": "user",
341                     "content": [
342                         {
343                             "type": "text",
344                             "text": f"""任务：为Markdown文档中的图片生成一个简短的
中文标题。
背景信息：
1. 所属文档标题: "{doc_title}"
2. 图片上下文:
{context_info}
请结合图片视觉内容和上述上下文信息，用中文简要总结这张图片的内容，
生成一个精准的中文标题（不要包含"图片"二字）。""",
351                             },
352                             {
353                                 "type": "image_url",
354                                 "image_url": {
355                                     "url": f"data:image/jpeg;base64,
{base64_image}"
356                                 }
357                             }
358                         ]
359                     }
360                 ],
361                 max_tokens=100,
362                 temperature=0.3
363             )
364         summary = response.choices[0].message.content.strip().replace("\n", "
")
365         return summary
366     except Exception as e:
367         logger.warning(f"图片摘要生成失败 {image_path}: {e}")
368         return "图片描述"
369 def _generate_image_summaries(
370     document_stem: str,
371     target_images_info: List[Tuple[str, str, Tuple[str, str, str]]],
372 ) -> Dict[str, str]:
373     """
374     调用多模态模型为图片生成内容摘要。
375
376     Args:
377         document_stem (str): 文档文件名（不含扩展名）。
378         target_images_info (List): 待处理图片信息列表。
379
380     Returns:
381         Dict[str, str]: 图片文件名到摘要的映射。
382     """
383     logger.info("step_3: 生成图片总结")
384
385     image_summaries = {}
386

```

```

387         # 初始化 VLM 客户端
388         try:
389             from openai import OpenAI
390             client = OpenAI(
391                 api_key="sk-ws-H.EPHYER.Zv9B.MEYCIQD00zNLzU-
SIYzg9R4PMcD3TAPx97i2M0P9PVbD7WNYAAIhA0kgkVI9r6W9zCBngcY-
DDk9SdwxGBQZzen7dQRAoCFT",
392                 base_url="https://llm-7tihr0u99gp8n6zv.cn-
beijing.maas.aliyuncs.com/compatible-mode/v1"
393             )
394         except Exception as e:
395             logger.error(f"初始化 VL 客户端失败: {e}")
396             return image_summaries
397
398         for image_filename, image_full_path, context_tuple in target_images_info:
399             logger.debug(f"正在生成摘要: {image_filename}")
400
401             summary_text = _call_vlm_for_summary(
402                 client,
403                 vl_model,
404                 image_full_path,
405                 document_stem,
406                 context_tuple
407             )
408             image_summaries[image_filename] = summary_text
409
410         return image_summaries
411
412     def _upload_images_and_replace_links(
413         minio_client,
414         document_stem: str,
415         target_images_info: List[Tuple[str, str, Tuple[str, str, str]]],
416         image_summaries: Dict[str, str],
417         md_content: str
418     ) -> str:
419         """
420         将图片上传至 MinIO 并替换 Markdown 中的本地链接。
421
422         Args:
423             minio_client: MinIO 客户端实例。
424             document_stem (str): 文档文件名。
425             target_images_info (List): 图片信息列表。
426             image_summaries (Dict): 图片摘要字典。
427             md_content (str): 原始 Markdown 内容。
428
429         Returns:
430             str: 替换链接后的 Markdown 内容。
431         """
432         logging.info("step_4: 上传图片并替换链接")
433
434         uploaded_urls = {}
435
436         # 遍历上传图片
437         for image_filename, image_full_path, _ in target_images_info:
438             object_name = f"{document_stem}/{image_filename}"
439
440             ext = os.path.splitext(image_full_path)[1].lower()

```

```

441         content_type = f"image/{ext[1:]}" if ext.startswith(".") else
"application/octet-stream"
442
443
444         base_protocol = "https://" if MINIO_SECURE else "http://"
445         minio_base_url = base_protocol + f"{MINIO_ENDPOINT}" + "/" + f"
{MINIO_BUCKET_NAME}"
446
447         if minio_client:
448             try:
449                 minio_client.fput_object(
450                     MINIO_BUCKET_NAME,
451                     object_name,
452                     image_full_path,
453                     content_type=content_type
454                 )
455                 remote_url = f"{minio_base_url}/{object_name}"
456                 uploaded_urls[image_filename] = remote_url
457                 logger.debug(f"图片上传成功: {image_filename} -> {remote_url}")
458             except Exception as e:
459                 logger.warning(f"图片上传失败 {image_filename}: {e}")
460         else:
461             logger.warning("MinIO 客户端未初始化, 跳过实际上传。")
462             uploaded_urls[image_filename] = \
463                 f"http://mock-minio/{document_stem}/{image_filename}"
464
465         # 替换 MD 中的链接
466         new_md_content = md_content
467         for image_filename, summary_text in image_summaries.items():
468             remote_url = uploaded_urls.get(image_filename)
469             if not remote_url:
470                 continue
471
472             replace_pattern = re.compile(
473                 r"!\[([.*?])\]\((.*?" + re.escape(image_filename) + r".*?)\)",
474                 re.IGNORECASE
475             )
476             new_md_content = replace_pattern.sub(
477                 f"![{summary_text}]({remote_url})",
478                 new_md_content
479             )
480
481         logger.info(f"成功替换了 {len(uploaded_urls)} 张图片的链接。")
482         return new_md_content
483
484     def _backup_new_md_file(
485         original_md_path_str: str,
486         new_md_content: str
487     ) -> str:
488         """
489         将处理后的 Markdown 内容写入新文件。
490
491         Args:
492             original_md_path_str (str): 原始文件路径。
493             new_md_content (str): 新的 Markdown 内容。
494
495         Returns:
496             str: 新文件的绝对路径。

```

```

497     """
498     logger.info("step_5: 备份新文件")
499
500     original_path = Path(original_md_path_str)
501     new_file_path = original_path.with_name(
502         f"{original_path.stem}_new{original_path.suffix}"
503     )
504
505     try:
506         with open(new_file_path, "w", encoding="utf-8") as f:
507             f.write(new_md_content)
508             logger.info(f"处理后的文件已备份至: {new_file_path}")
509     except Exception as e:
510         logger.error(f"写入新文件失败 {new_file_path}: {e}")
511
512     return str(new_file_path)
513
514 if __name__ == "__main__":
515     # # 1. pdf转换markdown
516     # test_pdf_path = Path(r".\knowledge_base\sample.pdf")
517     # test_output_dir = Path(r".\result")
518     # return_code = execute_mineru(test_pdf_path, test_output_dir)
519     # if return_code == 0:
520     #     md_path = _get_output_path(test_pdf_path, test_output_dir)
521     #     print(f"转换成功, 结果保存在: {md_path}")
522     # 2. 转换图片
523     md_path = r"result\sample\auto\sample.md"
524     # Step 1: 获取 Markdown 内容和相关路径
525     md_content, md_path_obj, images_dir_obj = _get_md_content_and_path(md_path)
526
527     # 如果没有 images 目录, 说明无需处理图片, 直接返回
528     if not images_dir_obj.exists():
529         logger.info("未找到 images 目录, 跳过图片处理流程。")
530     else:
531         # Step 2: 扫描并筛选需要处理的图片
532         target_images_info = _scan_and_filter_images(
533             md_content, images_dir_obj, image_extensions
534         )
535
536         if not target_images_info:
537             logger.info("未在 Markdown 中找到需要处理的有效图片引用。")
538
539         print(target_images_info)
540         # Step 3: 生成图片总结
541         image_summaries = _generate_image_summaries(
542             md_path_obj.stem,
543             target_images_info
544         )
545         print(image_summaries)
546         # 初始化 MinIO 客户端
547         minio_client = get_minio_client()
548
549         # Step 4: 上传图片并替换 Markdown 中的链接
550         new_md_content = _upload_images_and_replace_links(
551             minio_client,
552             md_path_obj.stem,
553             target_images_info,
554             image_summaries,

```

```
555         md_content
556     )
557     # Step 5: 备份生成新的 Markdown 文件
558     new_md_file_path = _backup_new_md_file(md_path, new_md_content)
559     print(f"处理完成, 结果保存在: {new_md_file_path}")
```